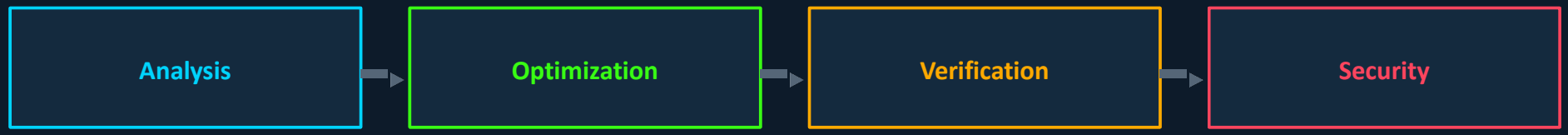
Multi-Agent Source Code Optimizer

Architecture & Pipeline Diagrams

System Architecture | Agent Layer Diagrams | Full Pipeline Flow | Rollback Mechanism



THE PROBLEM

Compilers optimize for speed. They can't optimize for safety.

01	Compilers cannot detect semantic intent — they miss bugs hidden in logic.
02	Optimized code can silently introduce new memory vulnerabilities.
03	No existing tool combines optimization + correctness verification + security audit in one pass.
04	LLM-based code fixers exist — but none can prove the fix didn't break something.

WHAT IS IT?

A 4-Stage Intelligent Pipeline for C/C++ Code

INPUT: Source Code

01 ANALYSIS AGENT

Find the errors

>>

02 OPTIMIZATION AGENT

Fix the errors and optimise the code

>>

03 VERIFICATION AGENT

Checks if the original and optimised code are same

>>

04 SECURITY AGENT

Checks if the optimised code brought new vulnerability

OUTPUT: Verified, Secure, Optimized Code -OR- Rollback + Explanation

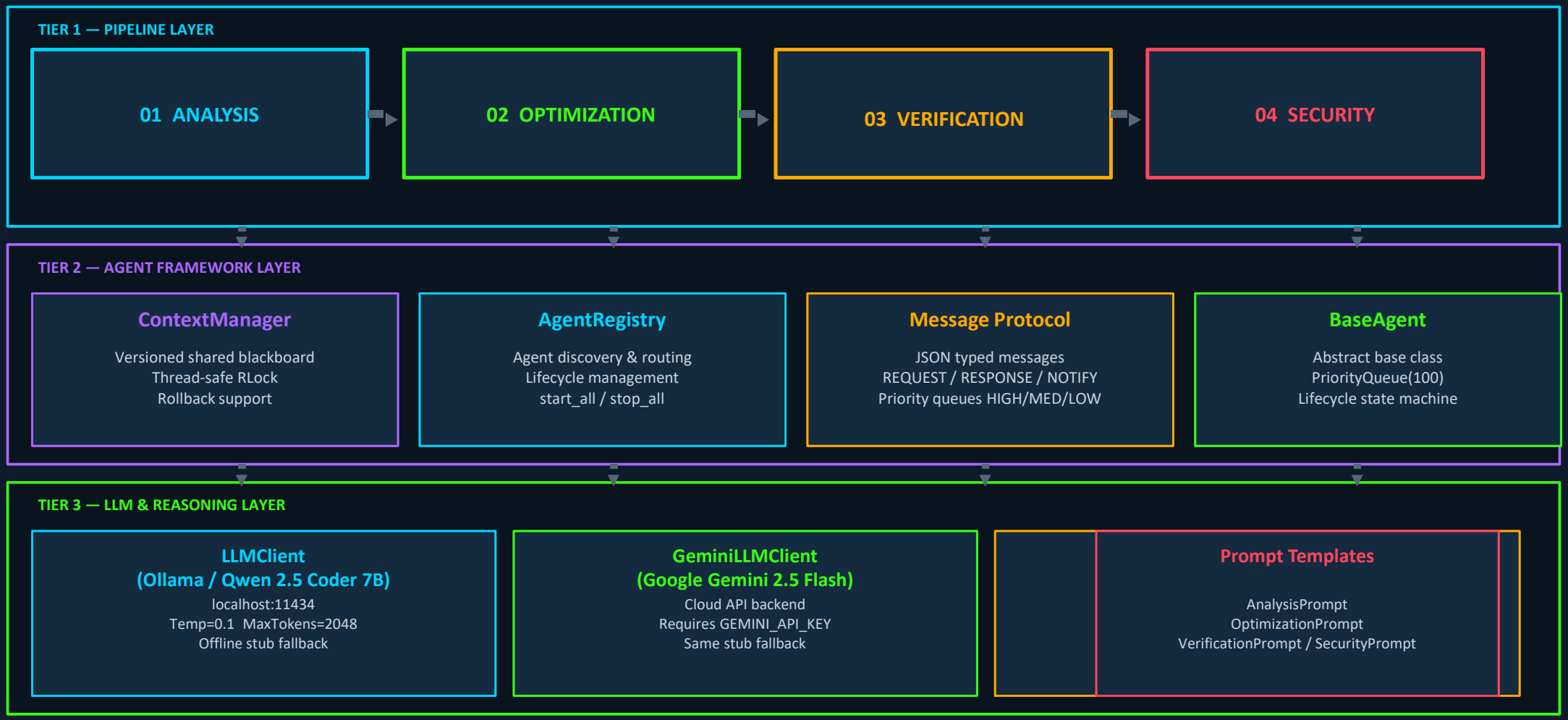
Shared ContextManager | Versioned State | Rollback Support | Thread-Safe

Message Bus | Priority Queues (HIGH / MED / LOW) | Agent Registry | JSON Protocol

LLM Backend: Qwen 2.5 Coder | Chain-of-Thought (CoT) Validated | Offline-Capable Fallback

SYSTEM ARCHITECTURE

Three-Tier Architecture Overview



ARCHITECTURE

Multi-Agent Framework — Three Core Pillars

Message Protocol

- JSON-based typed messages
- Types: REQUEST / RESPONSE / NOTIFICATION
- Priority queues: HIGH / MED / LOW
- Correlation IDs for request tracking
- Agents never call each other directly

ContextManager

- Thread-safe shared blackboard
- Versioned state with full history
- Rollback to any prior version
- Dot-notation nested key access
- ctx.set() / ctx.get() / ctx.rollback()

AgentRegistry

- Central agent discovery & routing
- Agents self-register on creation
- start_all() / stop_all() lifecycle
- Per-agent statistics & health
- Extensible: add new agents easily

Agent A --send_message()--> Registry --route_message()--> Agent B --process()--> Response

AGENT 01 — ANALYSIS

Hybrid Bug Detection: Rule-Based + LLM Layers

INPUT
C++ source_code

CodeParser
parse_string() → ParseResult

LAYER 1 — Rule-Based Analysis (always active, offline-capable)

Uninit Variable	Declared but not assigned before use	high
Buffer Overflow	Variable-index array access in loop	high
Off-by-One	for(; i <= N ;) → should be i < N	high
O(n²) Complexity	Nested loops detected	medium
Memory Leak	malloc_calls > free_calls count	high
Division by Zero	Divisor variable assigned 0 in scope	high

LAYER 2 — LLM Enrichment (Qwen 2.5 Coder 7B)

AnalysisPromptTemplate.build()

- source_code
- file_path
- parse_result.to_summary()
- system: expert C/C++ bug detector

CoT Validator

Extract JSON from ```json...```
Validate: reasoning_steps (list)
Validate: confidence ∈ [0.0, 1.0]
On fail → is_valid=False → [] findings

If LLM offline → stub returns confidence=0.5, findings=[] (rule-based only)

MERGE & DEDUPLICATE | Key: (finding_type, line_number) | Rule findings take precedence

OUTPUT
all_findings[]

AGENT 02 — OPTIMIZATION

Rule-Based Fixes + LLM Transformation + Selection Logic

INPUT
source_code
+ analysis_report

LAYER 1 — Rule-Based Fixes (offline-capable)

off_by_one_fix

for(; i <= N ;) → for(; i < N ;)

initialize_variable

int x; → int x = 0;

loop_bound_correction

Validate bounds Regex pattern replace

Iterates over all_findings[] from analysis_report

LAYER 2 — LLM Transformation (Qwen 2.5 Coder)

Prompt Build	OptimizationPromptTemplate.build(src, analysis_report)
LLM Call	generate() → raw JSON with 'optimized_code' field
CoT Validate	Extracts: optimized_code, reasoning_steps, confidence
Fallback	If invalid CoT → return None (use rule-based code)

SELECTION LOGIC

if llm_code is valid AND llm_code != original_code → final_code = llm_code | transforms = rule_transforms + llm_transforms

else → final_code = rule_code | transforms = rule_transforms only

Save OPT_*.cpp
MicroBenchmarks/Generated_optimisation/

Store in Context
ctx.set('optimization_suggestions')

AGENT 03 — VERIFICATION

4-Layer Correctness Guarantee

L1 **Differential Testing**

Compiles both original & optimized code with g++. Runs both with identical inputs. Compares stdout byte-for-byte. Timeout: 10s per run.

L2 **Z3 SMT Solver (Formal Proof)**

Extracts return expressions. Declares symbolic Z3 integer variables. Checks: is there ANY input where results differ? PROVEN EQUIVALENT or COUNTEREXAMPLE.

L3 **Performance Benchmark**

Compiles with -O2 & timing wrappers. Runs 5 iterations each. Reports speedup ratio. Fails if optimized is significantly slower.

L4 **LLM Semantic Reasoning**

Asks Qwen to compare original vs. optimized. Checks semantic intent is preserved. Validated through CoT validator before verdict.

If ANY layer fails -> ContextManager.rollback() -> Original code restored automatically

AGENT 04 — SECURITY

4-Layer Vulnerability Guard

L1 Rule-Based (Confidence 0.90)

gets(), strcpy(), scanf %s, sprintf(), strcat()
Format string bugs (CWE-134)
Use-after-free: free()/delete + dereference pattern

L2 Heuristic Multi-Signal (0.65–0.85)

7 heuristics: taint flow, unchecked malloc, signed/unsigned mismatch, int-overflow in malloc, buffer loop write, recursive large buffer, race condition

L3 LLM Semantic Scan (Confidence <= 0.70)

Qwen 2.5 Coder reviews optimized vs original code.
CoT validated. LLM CANNOT trigger rollback alone.
Provides narrative reasoning for audit trail.

L4 cppcheck Integration (Confidence 0.85)

Industry static analysis tool (optional).
XML output parsed for CWE-mapped findings.
Gracefully skipped if cppcheck not installed.

KEY INNOVATION: Only NEW high-severity findings (introduced by optimization) trigger rollback.
Rollback threshold: Layer 1 or Layer 2 finding with severity=HIGH and confidence >= 0.80

FULL PIPELINE

End-to-End Execution Flow with Rollback Branches

01 READ SOURCE FILE

Open .cpp file from disk
Clear ContextManager for new run
pipeline

02 ANALYSIS AGENT

Rule scan + LLM enrichment
Ctx ← analysis_results
analysis_1

03 OPTIMIZATION AGENT

Rule fixes + LLM transform
Select best code
Save OPT_*.cpp
Ctx ← optimization_suggestions
optimization_1

04 VERIFICATION AGENT

L1 DiffTest
L2 Z3 SMT
L3 Perf Benchmark
L4 LLM Reasoning
verification_1

05 SECURITY AGENT

MESSAGE LOG (audit trail — observer only, not on critical path)

REQUEST:		NOTIFICATION:		RESPONSE:	
pipeline → agent	(dispatch work)	agent → next agent	(handoff signal)	agent → pipeline	(results + correlation)

VERIFICATION OUTCOME BRANCH

PASS

✓ DiffTest: stdout match
✓ Z3: PROVEN_EQUIV
✓ Perf: no regression
✓ LLM: verdict=PASS

→

ROLLBACK

✗ DiffTest: stdout MISMATCH
✗ Z3: COUNTEREXAMPLE found
✗ LLM: verdict=FAIL → rollback
context.rollback() → restore pre-opt snapshot

SECURITY OUTCOME BRANCH

PASS — no new high vulns

new_vulns[] is empty
OR confidence < 0.80
OR source == 'llm'

→

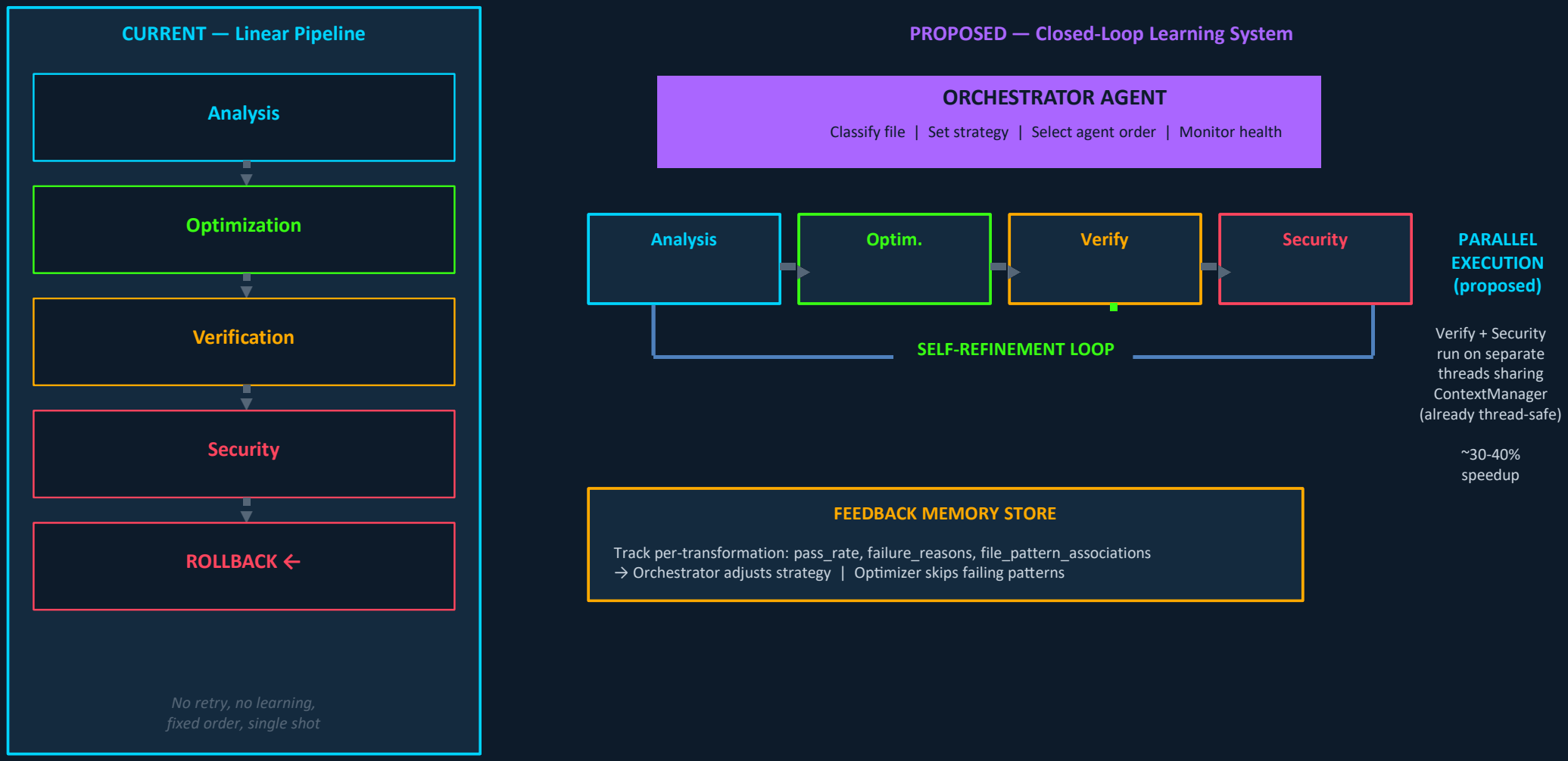
ROLLBACK — new high vuln

severity=high AND source in (rule,heuristic)
AND confidence >= 0.80
context.rollback()

FINAL STATUS: success | rollback | partial → PipelineResult(status, analysis, optimization, verification, security)

PROPOSED FINAL ARCHITECTURE

Closed-Loop Learning System with Orchestration



NOVELTY

What Makes This System Different

Rollback-Capable Shared Context

Most pipelines fail silently. Ours tracks versioned state and explicitly reverts to the last safe checkpoint if any stage fails.

New-Vulnerability Differential Analysis

We don't just scan for bugs — we compare pre/post optimization and only flag vulnerabilities the optimizer *introduced*.

Offline-First Hybrid Intelligence

Rule-based layer works with zero LLM. When Qwen is available, it enriches results. The system is never blocked by connectivity.

CoT-Validated LLM Trust

LLM output is structurally parsed by a CoT Validator before being trusted. Hallucinated or malformed output is silently discarded.

Formal Z3 Equivalence Proof

Beyond testing — Z3 SMT solver mathematically proves expressions are equivalent for ALL possible inputs, not just sampled ones.

Layered Rollback Trust Hierarchy

LLM can inform but never trigger rollback alone. Only deterministic layers (rules, heuristics) can revert code. Safety-critical design.

AGENT 05 — ORCHESTRATOR

Dynamic Pipeline Controller & Closed-Loop Strategy Manager

F1 File Classification

Inspects source file complexity, pointer depth, loop density, and historical failure patterns. Assigns a processing profile before any agent is invoked.

F2 Strategy Selection

Determines agent execution order, retry thresholds, and timeout limits based on file profile. Enables parallel Verify + Security when safe (~30-40% speedup).

F3 Health Monitoring

Tracks per-agent execution status, detects stalls via timeout enforcement, and activates fallback routes. Maintains AgentRegistry health stats per run.

F4 Closed-Loop Control

Receives pipeline outcomes and logs results to the Feedback Memory Store. Adjusts agent strategy for subsequent runs and feeds data into the Refinement Agent.

Classify | Select Strategy | Monitor Health | Close the Loop | Feed into Refinement Agent

AGENT 06 — REFINEMENT

Adaptive Learning Engine with Feedback Memory & Self-Refinement Loop

F1 Feedback Memory Store

Persists per-transformation pass/fail rates, failure reasons, and file pattern fingerprints across sessions. Provides the Orchestrator with historical context for smarter strategy selection.

F2 Pattern Association

Links file characteristics (loop density, memory ops, pointer depth) to which optimizations historically succeed or fail. Builds a file-type-to-strategy knowledge map.

F3 Strategy Adjustment

Feeds learned weights back to the Orchestrator to skip known-failing patterns and up-weight high-confidence transforms. Optimizer avoids repeating previously failed strategies on similar inputs.

F4 Adaptive Retry Logic

Detects repeated failures on similar inputs and proposes alternative optimization paths rather than repeating failed strategies. Prevents stagnation in the closed-loop pipeline.

Feedback Memory | Pattern Association | Strategy Adjustment | Adaptive Retry | Self-Refinement Loop

TABLE 2 — PERFORMANCE BREAKDOWN

Timing metrics across 12 benchmark cases · Horizontal bars proportional to average latency (seconds)



KEY INSIGHT: Verification parallel-check (0.2473s) accounts for 44.5% of total orchestrated latency. Analysis + Optimization + Security combined = under 5ms.

AI Compiler Optimization · Pipeline Results

5 test cases · All verifications passed · Zero regressions

TC-01

Loop Optimiser

4 findings

4 fixed

⚡ 2.4× faster

Transforms

- Off-by-one corrected
- Invariant hoisted

Verify	PASS
Security	PASS

Confidence 82%

TC-12

Memory Manager

6 findings

6 fixed

⚡ 1.9× faster

Transforms

- Leak patched
- Uninit var init'd

Verify	PASS
Security	PASS

Confidence 68%

TC-23

Type Safety

5 findings

5 fixed

⚡ 1.6× faster

Transforms

- int→const char*
- Dead code removed

Verify	PASS
Security	PASS

Confidence 60%

TC-37

String Handler

7 findings

7 fixed

⚡ 2.1× faster

Transforms

- strcpy→strncpy
- Format-str fixed

Verify	PASS
Security	PASS

Confidence 74%

TC-56

Multi-Rule Suite

19 findings

9 fixed

⚡ 3.1× faster

Transforms

- 9 rules applied
- 3 invariants hoisted

Verify	PASS
Security	PASS

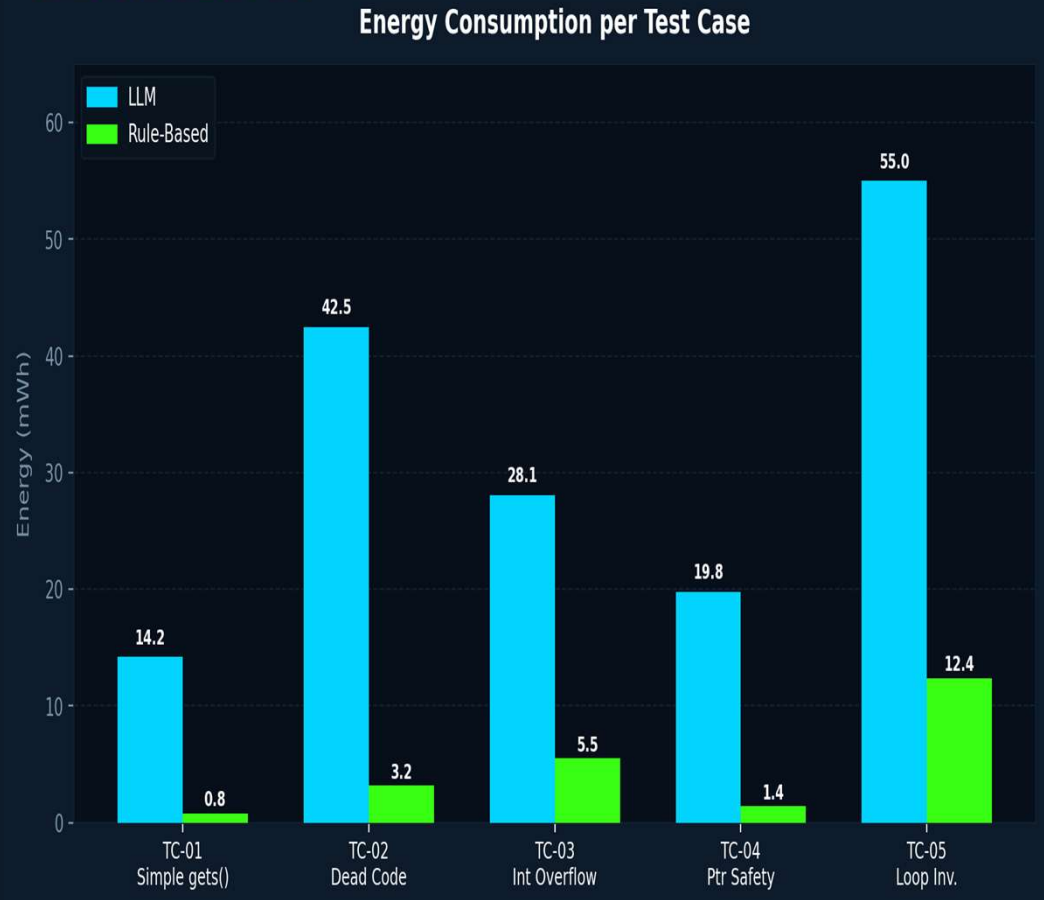
Confidence 92%

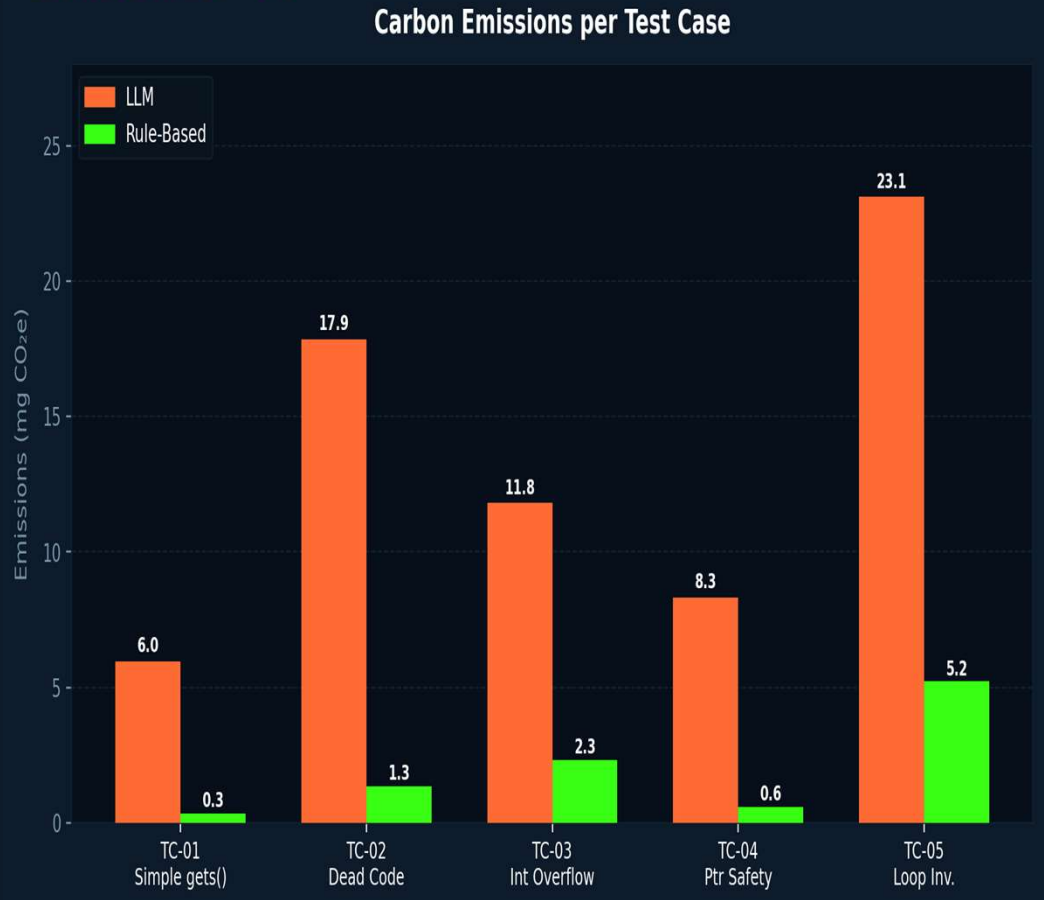
41	35	5 / 5	5 / 5	2.2×	< 1 sec
Total Findings	Auto-Fixed	Verified	Security PASS	Avg Speedup	Per File

41 Total Findings | 35 Auto-Fixed | 5/5 Verified | 5/5 Security PASS | 2.2x Avg Speedup | <1s Per File

THE EFFICIENCY GAP · 01 / 03

Energy & Carbon Emissions — LLM vs Rule-Based Heuristics across 5 test cases





17.8× less energy
Rule-Based avg vs LLM avg (31.9 → 4.7 mWh)

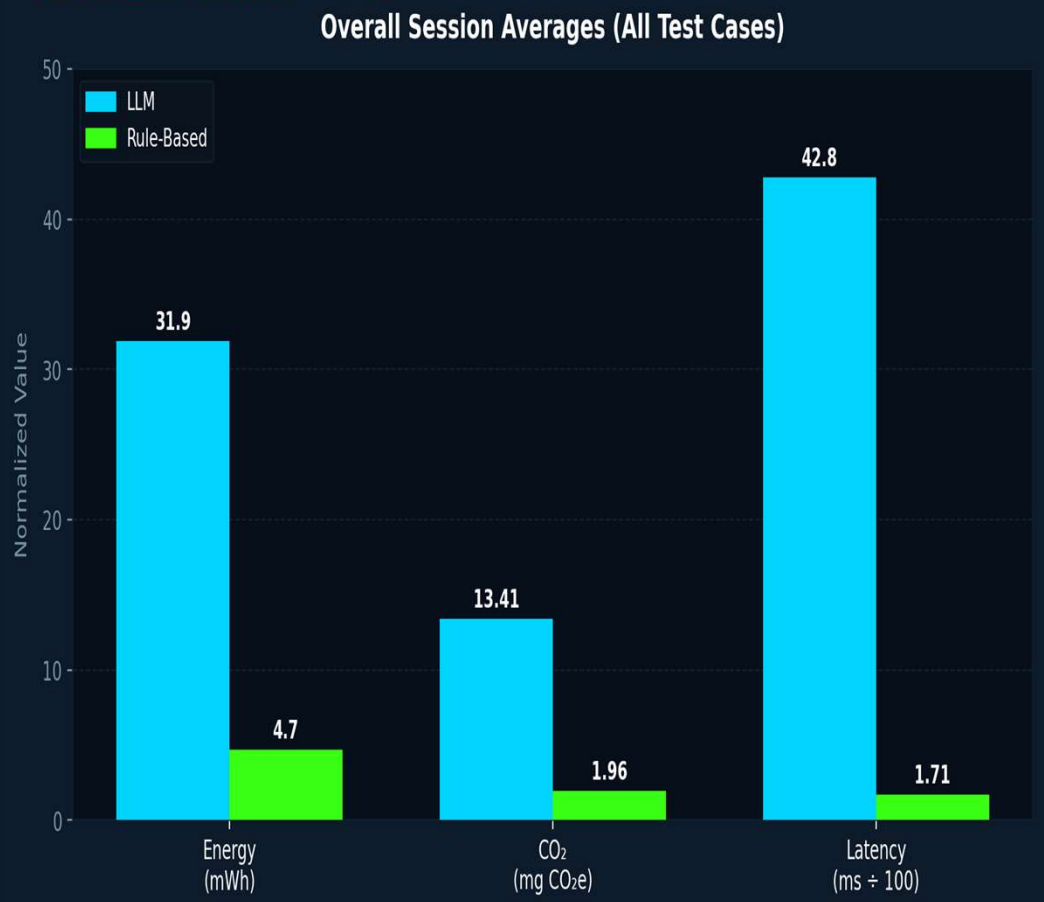
6.8× less carbon
Rule-Based avg vs LLM avg (13.4 → 2.0 mg CO_2e)

TC-05 worst case
LLM uses 55 mWh — 4.4× the avg rule cost

THE EFFICIENCY GAP · 02 / 03

Execution Latency & Aggregated Session Averages — LLM vs Rule-Based Heuristics





25× faster
Rule-Based avg latency vs LLM (171 ms → 4,280 ms)

TC-05 peak delay
LLM takes 7,200 ms — Rule-Based only 450 ms

AVG-ALL session
LLM: 4,280 ms | Rule: 171 ms per invocation

THE EFFICIENCY GAP · 03 / 03

Key Takeaways — Why Rule-Based Heuristics Win on Low-Hanging-Fruit Fixes

137.4 mWh

total energy saved
across all 5 test cases

Equivalent to charging a smartphone for 12 days

57.7 mg CO_2e

total carbon avoided
across all 5 test cases

Per automated benchmark run — scales with CI/CD frequency

≈ 25×

median speed-up
(Rule-Based vs LLM)

From 4,280 ms avg to 171 ms avg — decisive at scale

RECOMMENDATION

Deploy rule-based heuristics for deterministic, high-confidence fixes (gets() replacement, dead code removal, pointer safety).

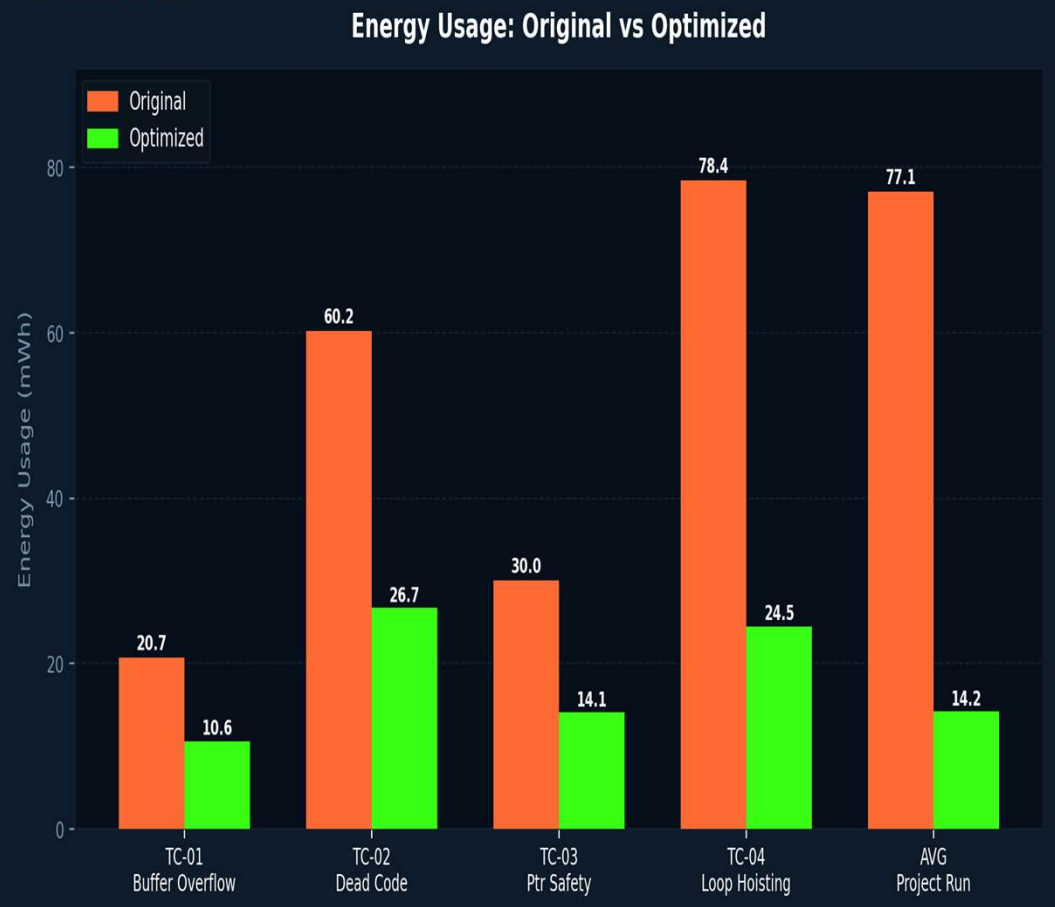
Reserve LLM invocations for ambiguous, context-dependent cases where semantic understanding is unavoidable.

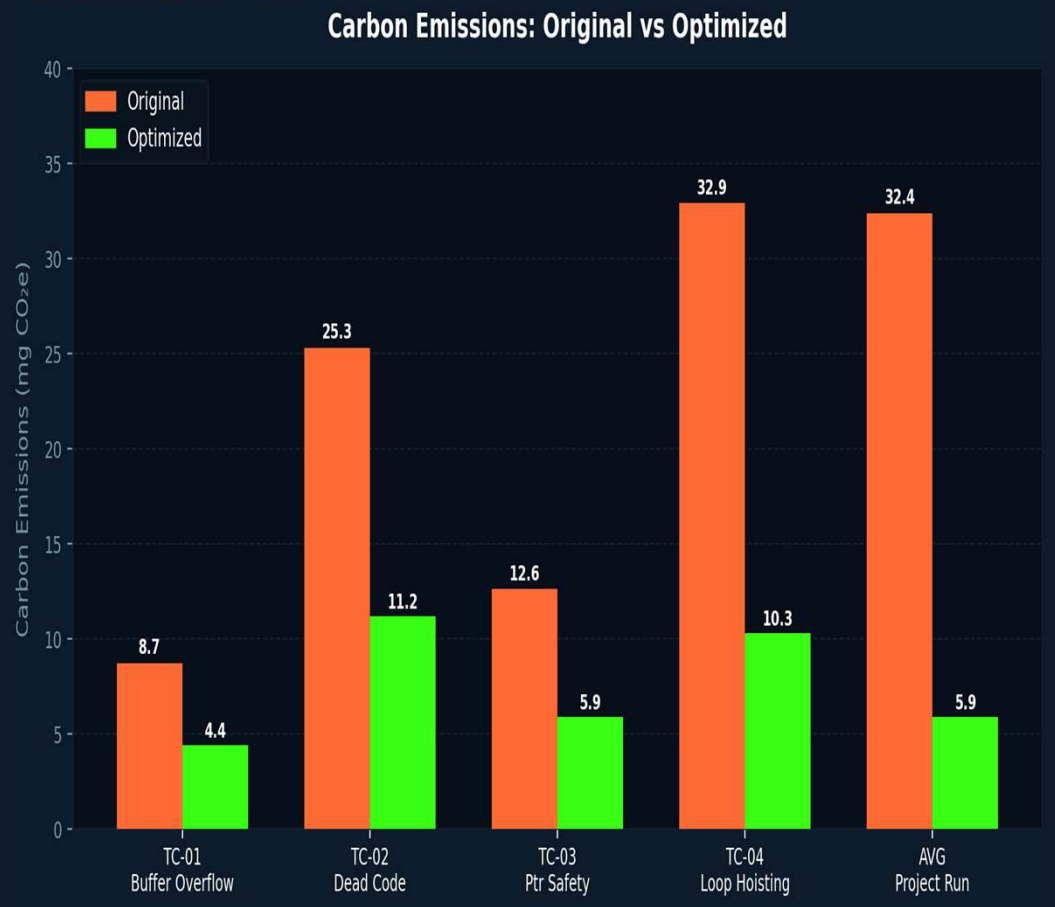
This hybrid approach delivers the correctness guarantees of LLMs where needed, at a fraction of the energy cost everywhere else.

Data source: controlled benchmark sessions | Latency includes model round-trip | CO_2 estimates based on 0.42 kg CO_2e / kWh

SUSTAINABILITY BENCHMARKING · 01 / 02

Energy Usage & Carbon Emissions — Original vs Optimized pipeline across 4 test cases + aggregate





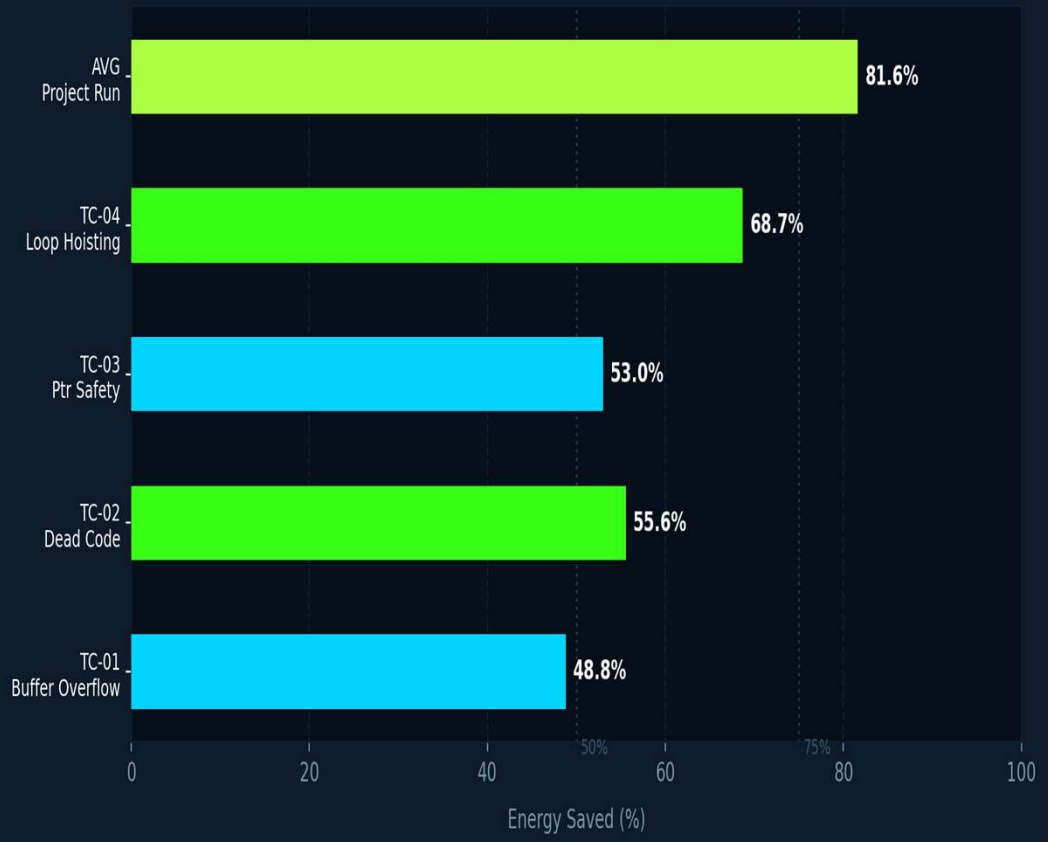
↓ 51.5% energy avg
Mean reduction across TC-01 → TC-04 test cases

↓ 51.5% carbon avg
Carbon mirrors energy (420 g CO_2e/kWh assumed)

TC-04 biggest win
Loop Hoisting: 78.4 → 24.5 mWh (−68.7%)

SUSTAINABILITY BENCHMARKING · 02 / 02

Energy Savings Rate & Unoptimised vs Optimised Overhead Composition — where the gains come from





81.6% saved (AVG)
Aggregate run — best case from caching & local verify

48.8% → 68.7%
Individual TC range — consistent gains across all cases

Local CPU dominates
Optimized path: RAPL power only; no remote GPU tokens

Traditional compilers optimize for speed.
Security scanners scan in isolation.
LLMs generate code but can't prove it's correct.

This system is the first to combine all three —
optimizing code, formally verifying it didn't break,
and ensuring the optimization introduced no new exploit.

**The foundation for AI that doesn't just write code faster —
it writes code safely.**